
Monotonicity as an Architectural Bias for Robust Language Models

Patrick Cooper¹ Alireza Nadali¹ Ashutosh Trivedi¹ Alvaro Velasquez¹

Abstract

Large language models (LLMs) are known to exhibit brittle behavior under adversarial prompts and jailbreak attacks, even after extensive alignment and fine-tuning. This fragility reflects a broader challenge of modern neural language models: small, carefully structured perturbations in high-dimensional input spaces can induce large and unpredictable changes in internal semantic representations and output.

We investigate *monotonicity* as an architectural inductive bias for improving the robustness of Transformer-based language models. Monotonicity constrains semantic transformations so that strengthening information, evidence, or constraints cannot lead to regressions in the corresponding internal representations. Such order-preserving behavior has long been exploited in control and safety-critical systems to simplify reasoning and improve robustness, but has traditionally been viewed as incompatible with the expressivity required by neural language models.

We show that this trade-off is not inherent. By enforcing monotonicity selectively in the feed-forward sublayers of sequence-to-sequence Transformers—while leaving attention mechanisms unconstrained—we obtain *monotone language models* that preserve the performance of their pre-trained counterparts. This architectural separation allows negation, contradiction, and contextual interactions to be introduced explicitly through attention, while ensuring that subsequent semantic refinement is order-preserving. Empirically, monotonicity substantially improves robustness: adversarial attack success rates drop from approximately 69% to 19%, while standard summarization performance degrades only marginally. We further show that the robustness benefit transfers

to a decoder-only foundation model (Pythia-1.4B), but only when the constraint scope is matched to the architecture: constraining the feed-forward input projection alone—sufficient in the encoder-decoder setting—fails outright, whereas constraining both feed-forward projections reduces the mean attack success rate from 69.0% to 42.5% across three seeds, at a larger cost in language-modeling perplexity. Monotonicity is thus a practical inductive bias for robustness, with an architecture-dependent quality–robustness trade-off.

1. Introduction

Large language models (LLMs) have demonstrated remarkable capabilities across a wide range of natural language tasks, yet their behavior under adversarial or carefully structured inputs remains brittle. Even models that have undergone extensive alignment and fine-tuning can be induced to produce unsafe or unintended outputs through jailbreak prompts or small, targeted perturbations. This phenomenon is now well documented and points to a deeper underlying issue: modern language models operate in extremely high-dimensional spaces, where even subtle changes to the input or internal representations can induce large and difficult-to-predict changes in the output. Recent work has attributed this sensitivity, in part, to the high Lipschitz constants exhibited by such models, which amplify perturbations as they propagate through the network (Newhouse et al., 2025).

This fragility has motivated a line of research on improving the robustness of language models (Salah, 2026; Su et al., 2024). Existing approaches largely focus on training-time interventions, such as alignment objectives (Cao et al., 2024) or adversarial training (Howe et al., 2024), as well as inference-time defenses such as output filtering (Khachaturov & Mullins, 2025). While these methods can be effective empirically, they are inherently reactive, addressing vulnerabilities after they are discovered rather than constraining model behavior by design. As language models are increasingly deployed, this raises a question: can we endow language models with structural properties that make their behavior more predictable under perturbation?

^{*}Equal contribution ¹Department of Computer Science, University of Colorado Boulder, Boulder, CO, USA. Correspondence to: Patrick Cooper <patrick.cooper@colorado.edu>, Alireza Nadali <alireza.nadali@colorado.edu>.

We investigate *monotonicity* as a structural inductive bias for improving the robustness of language models. At a high level, monotonicity constrains a model so that its outputs change in a predictable direction with respect to changes in its inputs. This property has a long history in machine learning (Gupta et al., 2016; Runje & Shankaranarayana, 2023) and related fields (Smith, 1995), and has been especially influential in control theory, where monotone systems admit strong guarantees on stability and safety even in very high-dimensional settings (Michel et al., 2015; Ito et al., 2014). For example, monotonicity has been used to analyze and certify the behavior of large-scale dynamical systems such as power grids, where direct reasoning over all possible system states is infeasible (Rantzer & Bernhardsson, 2014).

Beyond its role in classical dynamical systems, monotonicity also aligns naturally with how we expect language models to behave at the level of semantic transformation. Many linguistic and reasoning operations are inherently order-preserving: adding supporting evidence should not weaken a conclusion; adding constraints or safety instructions should not result in less constrained internal representations; and increasing the stated severity or risk of a situation should not lead to a more permissive response. Yet contemporary language models frequently violate these expectations, exhibiting behavior in which the addition of seemingly clarifying or restrictive information leads to unpredictable or regressive changes in output.

Concrete examples of semantic monotonicity arise naturally in language understanding. Consider a sequence of prompts that progressively strengthen a factual claim (“*The medication has side effects*” $\preceq$ “*The medication has severe side effects*” $\preceq$ “*The medication has severe side effects and requires medical supervision*”), or that add explicit safety constraints (“*Explain this process*” $\preceq$ “*Explain this process safely*” $\preceq$ “*Explain this process safely without providing actionable steps*”). Semantically, such prompts form a partial order: later prompts contain strictly more information or stricter constraints. A model’s internal representations should reflect this ordering, and its outputs should respect it. When models fail to do so, the cause is often not the absence of relevant information, but the way semantic features are transformed and recombined internally.

This perspective highlights a structural source of brittleness in modern architectures. Transformer models interleave contextual aggregation, via attention, with nonlinear feed-forward transformations that refine token-level representations. While attention mechanisms are well suited to introducing contextual operators such as negation, contrast, or exception by explicitly linking tokens and defining scope, the subsequent feed-forward transformations are unconstrained and may arbitrarily amplify, suppress, or cancel features. As a result, semantic reversals can occur implicitly

through distributed feature cancellation rather than being driven by explicit linguistic signals. We argue that this implicit cancellation is a key contributor to brittle behavior under adversarial or carefully structured prompts.

Our motivation for monotonicity draws a close conceptual parallel to *monotone Boolean circuits* (Jukna et al., 2012). In monotone circuits, logical negation is disallowed within the circuit itself; any negation must be introduced explicitly at the inputs. This restriction does not eliminate expressive power, but enforces a semantic discipline: truth cannot be undone by downstream computation. Analogously, enforcing monotonicity in semantic transformations encourages language models to represent negation, contradiction, or constraint relaxation explicitly through contextual mechanisms such as attention, rather than implicitly through fragile cancellations inside nonlinear transformations. Monotonicity thus serves as a design principle that separates *where semantic reversals may occur* from *where meaning is refined*, yielding predictable, auditable, and robust representations.

Viewed through this lens, a language model can be understood as a high-dimensional system whose internal semantic state evolves through successive transformations in response to input perturbations. Imposing monotonic structure on these transformations limits how adversarial variations propagate, ensuring that strengthening information or constraints cannot lead to regressions in meaning. As in monotone dynamical systems, this structure simplifies reasoning about global behavior. In such settings, it suffices to verify satisfaction only at extreme points (Feyzmahdavian et al., 2017).

Our central finding is that monotonicity can be incorporated into Transformer architectures without sacrificing performance by enforcing it selectively within feed-forward sublayers, while leaving attention mechanisms unconstrained. This design preserves expressivity while imposing a disciplined form of semantic refinement. The resulting monotone Transformers match the task performance of their non-monotone counterparts and exhibit substantially improved robustness to adversarial and jailbreak attacks. In particular, these gains arise purely from architectural structure, without additional data, modified training objectives, or heuristic defenses, isolating monotonicity itself as a causal factor.

These results challenge the prevailing view, articulated in prior studies of constrained monotonic networks (Sartor et al., 2025), that enforcing strong architectural constraints inevitably degrades optimization or expressive capacity in large neural models. Instead, they suggest that monotonicity can serve as a practical and principled inductive bias for shaping semantic behavior. Motivated by this perspective, we investigate how monotonicity can be imposed as a structural property of language models in a manner that is both semantically meaningful and practically viable. In particular, we focus on enforcing monotonicity at the level

of semantic refinement, rather than as a global constraint, and develop this approach through both formal analysis and empirical evaluation.

Contributions. Our contributions are as follows:

1. **Monotone Transformer components.** We introduce monotone feed-forward sublayers for Transformers, enforcing structural monotonicity while leaving attention mechanisms unconstrained.
2. **Lossless distillation.** We show that pretrained Transformers can be distilled into monotone counterparts without degrading standard benchmark performance.
3. **Improved robustness.** We empirically demonstrate that monotone language models exhibit increased robustness to adversarial and jailbreak attacks.
4. **Architecture-conditional transfer.** We characterize how the robustness benefit of monotonicity transfers from encoder-decoder to decoder-only architectures, showing that decoder-only models require broader constraint coverage to close adversarial leak paths, and quantifying the resulting quality–robustness trade-off on Pythia-1.4B.

2. Related Work

Vulnerabilities of Language Models. Large language models are highly susceptible to adversarial manipulations that bypass alignment and safety mechanisms. Prior work has documented jailbreak prompts and transferable adversarial suffixes that generalize across models and settings, including aligned systems (Zou et al., 2023). Subsequent studies identify additional failure modes, such as position-sensitive jailbreaks (Wang et al., 2025b), interpretable attacks that evade perplexity-based defenses (Zhu et al., 2023), and many-shot jailbreaks whose effectiveness increases with larger context windows (Anil et al., 2024). Robustness benchmarks further show that models perform poorly under systematic adversarial evaluation (Wang et al., 2021), and that safety mechanisms often fail in multilingual or low-resource settings (Deng et al., 2024). Prompt injection attacks, which exploit the lack of separation between instruction and context, are now recognized as a fundamental vulnerability, alongside broader threats to inference and training-time surveyed in recent security analyses (Li & Fung, 2025).

Defenses and Robustness Enhancements. Proposed defenses include smoothing-based methods with provable jailbreak guarantees (Robey et al., 2023), lightweight detection based on output distributions (Sayeedi et al., 2025), and adversarial training approaches whose effectiveness depends on model scale and threat model (Howe et al., 2024). Other work examines attack transferability and adaptive prompt

sequences to harden models against structured adversarial dependencies (Wang et al., 2025a). Despite these advances, most defenses remain reactive and training-dependent.

Monotonic Neural Networks and Structural Constraints.

Monotonicity has long been studied as a structural inductive bias for interpretability and robustness. Classical results show that feed-forward networks with non-negative weights and monotone activations are universal approximators of monotone functions, establishing that monotonicity need not limit expressivity (Sill, 1998; Daniels & Velikova, 2010). Subsequent work proposed practical monotone and partially monotone architectures, including lattice-based models, enabling scalable learning under monotonic constraints in structured domains (Gupta et al., 2016; You et al., 2017). More recent research highlights monotonicity’s benefits for verification, reducing reasoning to boundary inputs in piecewise linear networks (Weber et al., 2021). More broadly, architectural constraints and structured layers have been shown to improve robustness and stability without prohibitive performance loss (Amos & Kolter, 2017). Our work builds on this line by showing that selectively enforced monotonicity can scale to Transformer-based language models and yield tangible robustness gains.

3. Monotone Transformers

We study the role of monotonicity constraints in improving the adversarial robustness of sequence-to-sequence (Seq2Seq) language models.

Definition 3.1 (Sequence-to-Sequence Model). A sequence-to-sequence model is a parameterized mapping

$$\mathcal{M}_\theta : \mathcal{X} \rightarrow \mathcal{Y},$$

where $\mathcal{X}$ denotes the space of input token sequences and $\mathcal{Y}$ denotes the space of output token sequences. In this work, $\mathcal{M}_\theta$ is instantiated as a Transformer architecture composed of attention layers, feed-forward network (FFN) sublayers, residual connections, and normalization operators.

Fix $A \in \mathbb{R}^{p \times d}$ whose rows define interpretable semantic axes. For a hidden state $h \in \mathbb{R}^d$, define semantic coordinates $s := Ah \in \mathbb{R}^p$ and equip $\mathbb{R}^p$ with the standard product order. We define a preorder on $\mathbb{R}^d$ by

$$h \preceq h' \iff Ah \leq Ah' \text{ (elementwise),}$$

and interpret $h' \succeq h$ as semantically stronger along the chosen axes. In practice, we choose A either by (i) training p linear probes on curated ordered prompt pairs and using their weights as rows of A , or (ii) taking a small set of fixed directions from auxiliary classifiers; in both cases, A is fixed before imposing monotonicity constraints. When $A = I$, this reduces to the coordinatewise order.

Enforcing monotonicity across all d hidden dimensions would require every FFN weight to be nonnegative, severely limiting expressivity. Instead, A selects a p -dimensional semantic subspace ($p \ll d$) along which monotonicity holds, leaving the remaining $d - p$ directions in $\ker(A)$ unconstrained. Each monotone sublayer projects via Ah , applies the constrained MLP g in $\mathbb{R}^p$, and lifts back via $A^\dagger$. The identity $AA^\dagger = I_p$ reduces the semantic-space map to $T(s) = s + g(s)$, so monotonicity in $\mathbb{R}^p$ follows directly from that of g without requiring the full Jacobian $J_F(h)$ to be nonnegative. In our experiments we use $p = 64$ with $d = 512$, constructing A from probe weight vectors trained on ordered prompt pairs. A and $A^\dagger$ are frozen before constrained training begins.

Definition 3.2 (Monotonicity w.r.t. $\preceq$). A function $F : \mathbb{R}^d \rightarrow \mathbb{R}^d$ is A -monotone if

$$\begin{aligned} h \preceq h' &\Rightarrow F(h) \preceq F(h'), \\ \text{equivalently, } Ah \leq Ah' &\Rightarrow AF(h) \leq AF(h'). \end{aligned}$$

Semantic-monotone FFN sublayer. Assume A has full row rank and let $A^\dagger \in \mathbb{R}^{d \times p}$ denote a fixed right inverse of A (e.g., the Moore–Penrose pseudoinverse), so that $AA^\dagger = I_p$.

Define an FFN update that acts in semantic coordinates:

$$F(h) := h + A^\dagger g(Ah),$$

where $g : \mathbb{R}^p \rightarrow \mathbb{R}^p$ is an ordinary coordinatewise-monotone MLP.

Proposition 3.3 (Sufficient condition for A -monotonicity). *If g is monotone w.r.t. the product order on $\mathbb{R}^p$, then F is A -monotone. In particular, it suffices that g is an MLP with elementwise non-decreasing activations and elementwise nonnegative weight matrices.*

Proof. If $Ah \leq Ah'$, monotonicity of g gives $g(Ah) \leq g(Ah')$, and

$$\begin{aligned} AF(h) &= Ah + AA^\dagger g(Ah) \\ &= Ah + g(Ah) \\ &\leq Ah' + g(Ah') \\ &= AF(h'). \end{aligned}$$

□

Definition 3.4 (Monotonicity). A function $f : \mathbb{R}^n \rightarrow \mathbb{R}^m$ is *monotone* if for all $x, x' \in \mathbb{R}^n$,

$$x \leq x' \Rightarrow f(x) \leq f(x').$$

Lemma 3.5 (Closure under Composition). *If $f : \mathbb{R}^n \rightarrow \mathbb{R}^m$ and $g : \mathbb{R}^m \rightarrow \mathbb{R}^k$ are monotone, then their composition $g \circ f$ is monotone.*

Consider a feed-forward neural network $N : \mathbb{R}^{n_0} \rightarrow \mathbb{R}^{n_k}$ with k hidden layers. For input $u \in \mathbb{R}^{n_0}$,

$$\begin{aligned} y_0 &= u, \\ y_{i+1} &= \sigma(W_i y_i + b_i), \quad i = 0, \dots, k-1, \\ N(u) &= W_k y_k + b_k, \end{aligned}$$

where σ is applied elementwise.

Proposition 3.6 (Sufficient Condition for FFN Monotonicity). *If σ is elementwise non-decreasing and all weight matrices satisfy*

$$W_i \geq 0 \quad \text{for } i = 0, \dots, k,$$

then N is monotone.

Proof. Each affine map with nonnegative weights is monotone, and σ preserves monotonicity by assumption. The claim follows by Lemma 3.5. □

Neural networks satisfying Proposition 3.6 are commonly referred to as *monotone neural networks*. Such networks are universal approximators of *continuous monotone functions* on compact domains (Daniels & Velikova, 2010).

Definition 3.7 (Monotone Transformer). A Transformer model $\mathcal{M}_\theta$ is A -monotone with respect to its feed-forward sublayers if each FFN sublayer implements an update of the form

$$F(h) := h + A^\dagger g(Ah),$$

where $A^\dagger \in \mathbb{R}^{d \times p}$ is a fixed right inverse of A satisfying $AA^\dagger = I_p$ (e.g., $A^\dagger = A^\top (AA^\top)^{-1}$ when A has full row rank) and $g : \mathbb{R}^p \rightarrow \mathbb{R}^p$ is coordinatewise monotone (e.g., an MLP with elementwise non-decreasing activations and elementwise nonnegative weight matrices). All other components (attention, residuals, normalization) are left unconstrained.

This definition does not assert global monotonicity of the full Transformer, but enforces monotonic semantic refinement at the level of FFN sublayers, where implicit feature cancellation is most likely to occur (Runje & Shankaranarayana, 2023; Sartor et al., 2025; Nguyen et al., 2023).

We enforce coordinatewise monotonicity of $g : \mathbb{R}^p \rightarrow \mathbb{R}^p$ by constraining all linear maps inside g to be elementwise nonnegative and using an elementwise non-decreasing activation σ . Concretely, for each constrained weight matrix $W \in \mathbb{R}^{m \times n}$ in g , we reparameterize

$$W = \phi(V), \quad \phi(v) = \log(1 + \exp(v)),$$

with unconstrained V , applied elementwise. This guarantees $W \geq 0$ without projection or constrained optimization.

We fix A (and hence $A^\dagger$) before training. We initialize g to be near the zero map so that $F(h) \approx h$ at the start

of distillation/fine-tuning. Specifically, we initialize the unconstrained parameters V so that $\phi(V)$ is small (e.g., $V \ll 0$ elementwise), and initialize biases to zero.

Rather than projecting after each update, we maintain $W \geq 0$ throughout optimization via the softplus reparameterization above, ensuring g remains coordinatewise monotone during training.

4. Experimental Setup

We instantiate the monotone Transformer framework of Section 3 using the T5 architecture (Raffel et al., 2020), enabling a controlled empirical study of monotonicity as an architectural inductive bias for robustness.

Implementation Scope. We replace each Transformer FFN sublayer by the semantically-monotone update

$$F(h) := h + A^\dagger g(Ah),$$

where $A \in \mathbb{R}^{p \times d}$ is fixed, $A^\dagger$ satisfies $AA^\dagger = I_p$, and $g : \mathbb{R}^p \rightarrow \mathbb{R}^p$ is a coordinatewise-monotone MLP enforced via elementwise nonnegative weights and monotone activations. Attention layers, residual connections, and normalization remain unconstrained.

We use T5-small, comprising 6 encoder and 6 decoder layers with 512-dimensional hidden representations, 8 attention heads per layer, and 2048-dimensional FFN intermediate activations. Of the model’s approximately 60M parameters, roughly 24M (40%) correspond to the FFN sublayers that we replace with the semantic-space update; within each such sublayer, only the parameters inside $g : \mathbb{R}^p \rightarrow \mathbb{R}^p$ are constrained to be elementwise nonnegative to enforce coordinatewise monotonicity in semantic space.

Training and evaluation protocols, including optimization hyperparameters, datasets, and decoding settings, are detailed in Appendix A.2.

4.1. Adversarial Robustness Evaluation

We evaluate robustness under two complementary classes of adversarial attacks designed to probe different failure modes of sequence-to-sequence models.

Universal Adversarial Triggers. Universal adversarial triggers (UATs) (Wallace et al., 2019) consist of short, input-agnostic token sequences that, when prepended to an input, maximize the model’s loss. We optimize triggers using coordinate-wise search with three random restarts and 50 iterations. Candidate tokens are drawn from a vocabulary biased toward punctuation, special characters, and high-frequency words that empirically induce greater disruption. Triggers are optimized on a held-out validation split and evaluated on a disjoint test set. To assess transferability, we additionally construct a transfer matrix measuring cross-

model vulnerability to learned triggers.

HotFlip Attacks. HotFlip attacks (Ebrahimi et al., 2018) identify vulnerable input positions by computing gradients of the loss with respect to token embeddings and replacing selected tokens to maximize loss increase. We allow up to five token replacements per example, selecting replacements based on the dot product between gradient directions and candidate embeddings.

Robustness Metrics. Robustness is quantified using ROUGE score degradation under attack, attack success rate (defined as a ROUGE-L degradation exceeding 10%), and statistical significance assessed via independent t -tests. We additionally report mean loss increase and analyze cross-model transferability of adversarial triggers.

4.2. Decoder-Only Foundation Model Track

The T5 experiments above concern an encoder-decoder architecture. To determine whether the monotonicity-robustness relationship generalizes to the decoder-only architectures that dominate contemporary foundation models, we conduct a second experimental track using Pythia-1.4B (Biderman et al., 2023), a 24-layer autoregressive language model from the GPT-NeoX family. Models are fine-tuned and evaluated on the Pile (Gao et al., 2020) (uncopyrighted subset), and clean performance is measured by *perplexity*—the exponentiated mean negative log-likelihood per token—on a held-out sample, the standard intrinsic quality metric for autoregressive language models.

Because decoder-only blocks lack the encoder-decoder bottleneck of T5, it is not obvious which sublayers must be constrained for monotonicity to bind. Each GPT-NeoX block contains an attention sublayer with an output projection and a feed-forward (MLP) sublayer with an input projection ($W_{\text{in}} \in \mathbb{R}^{4d \times d}$) and an output projection ($W_{\text{out}} \in \mathbb{R}^{d \times 4d}$). We therefore compare three *constraint scopes*, each applying the softplus reparameterization of Section 3 directly to the designated weight matrices in all 24 layers (enforcing elementwise nonnegativity, i.e., the coordinatewise order):

- **MLP-in:** constrain only W_{in} , the direct analogue of the T5 scheme;
- **MLP-both:** constrain both W_{in} and W_{out} ;
- **MLP-in + Attn-out:** constrain W_{in} and the attention output projection.

After imposing constraints, each model undergoes *recovery training*—fine-tuning on the Pile to re-establish effective representations under the constrained parameterization—with the unconstrained baseline fine-tuned under an identical protocol. Constraint scopes are first screened under a reduced training budget on a single seed; the selected

scope is then validated under the full recovery budget (5 epochs baseline, 10 epochs monotonic) across three seeds (42, 1337, 2024). Robustness is assessed with the same attack implementations as the T5 track: HotFlip with $n = 200$ examples and up to 10 token flips, and UATs optimized on 80 texts and evaluated on 120 disjoint texts, with attack effect measured as the relative increase in negative log-likelihood (NLL) under the learned trigger.

5. Mechanisms of Gradient Attenuation in Monotone Feed-Forward Networks

We provide a mechanistic explanation for the empirical robustness gains observed in monotone Transformer language models. Rather than claiming global robustness guarantees, we analyze how monotonicity constraints alter the local gradient structure of feed-forward sublayers in a way that can reduce the effectiveness of gradient-based adversarial attacks such as HotFlip.

Our analysis is motivated by classical results in the theory of monotone dynamical systems, which show that, under mild boundedness assumptions, trajectories of strongly monotone flows converge to equilibrium points almost surely; in particular, derivatives along these trajectories tend to zero as the system approaches an invariant set (Hirsch, 1985). While a feed-forward network is not a time-indexed dynamical system in this sense, this perspective suggests that monotonicity can limit the set of active directions along which outputs and gradients vary. We leverage this intuition to inform a local analysis of gradient behavior under monotone perturbations, focusing on how saturation and non-negative weights interact to diminish exploitable gradient directions.

Consider an A -monotone FFN update

$$F(h) := h + A^\dagger g(Ah),$$

where $A \in \mathbb{R}^{p \times d}$ is fixed, $A^\dagger$ satisfies $AA^\dagger = I_p$, and $g : \mathbb{R}^p \rightarrow \mathbb{R}^p$ is coordinatewise monotone (e.g., an MLP with elementwise non-decreasing activations and elementwise nonnegative weight matrices). Let $s := Ah \in \mathbb{R}^p$ denote semantic coordinates and define the induced semantic refinement map

$$T(s) := AF(h) = s + g(s).$$

We analyze the Jacobian of T and its implications for gradients with respect to semantic coordinates. (In general, $J_F(h)$ need not be elementwise nonnegative in $\mathbb{R}^d$ due to $A^\dagger$.)

Lemma 5.1 (Non-negativity of the Semantic Jacobian). *If g is implemented as an MLP with elementwise non-decreasing activations and elementwise nonnegative weight matrices, then the Jacobian*

$$J_T(s) = \nabla_s T(s)$$

has non-negative entries for all $s \in \mathbb{R}^p$.

This follows from $T(s) = s + g(s)$ and the factorization of the MLP Jacobian. For a two-layer $g(s) = W_2 \sigma(W_1 s + b_1) + b_2$ with $W_1, W_2 \geq 0$ elementwise,

$$J_g(s) = W_2 \text{diag}(\sigma'(W_1 s + b_1)) W_1 \geq 0 \quad (\text{elementwise}),$$

hence $J_T(s) = I + J_g(s) \geq 0$ elementwise. Intuitively, increasing any semantic coordinate cannot decrease any output semantic coordinate through T ; sign cancellations are ruled out *within semantic space*.

Let $\mathcal{L} : \mathbb{R}^p \rightarrow \mathbb{R}$ be any differentiable scalar objective defined on semantic coordinates (e.g., $\mathcal{L}(s')$ induced by the rest of the network through $s' = Ah'$). By the chain rule,

$$\nabla_s \mathcal{L}(T(s)) = J_T(s)^\top \nabla_{s'} \mathcal{L}(s')|_{s'=T(s)}.$$

Thus the backpropagated gradient in semantic space is filtered by $J_T(s)^\top$, whose entries are nonnegative under Lemma 5.1.

5.1. Saturation-Induced Gradient Attenuation

We now examine how saturation in monotone networks affects gradient magnitude.

Assumption 5.2 (Saturating Activations). The activation σ used inside g has bounded derivative and admits saturated regimes, i.e., $\sigma'(z)$ becomes arbitrarily small as $z \rightarrow +\infty$.

Lemma 5.3 (Gradient Attenuation under Saturation). *Let $T(s) = s + g(s)$ where g is an MLP with elementwise nonnegative weights and activation σ satisfying Assumption 5.2. Suppose that along a sequence $\{s_k\}$, a non-empty subset of pre-activations in g diverges to $+\infty$, so that the corresponding entries of σ' converge to 0. Then the contribution of those saturated units to $J_g(s_k)$ vanishes, and hence their contribution to $\nabla_s \mathcal{L}(T(s_k)) = J_T(s_k)^\top \nabla_{s'} \mathcal{L}$ vanishes asymptotically.*

Proof is deferred to Appendix A.1.2.

This result shows that monotonicity, when combined with activation saturation, can locally attenuate gradient magnitude without eliminating all gradient signal.

5.2. Persistence of Saturation Effects

The following observation captures a one-sided stability property of saturated units under monotone perturbations.

Lemma 5.4 (Persistence of Saturated Units). *Let $s \leq s'$ and consider any first-layer pre-activation $z(s) = (W_1 s + b_1)_j$ within g , with $W_1 \geq 0$ elementwise. If unit j is saturated at s' , i.e., $\sigma'(z(s')) = 0$, then for any monotone semantic perturbation $\delta \geq 0$, the unit remains saturated at $s' + \delta$.*

Proof is deferred to Appendix A.1.3.

Importantly, this lemma applies to individual hidden units rather than to the entire input gradient, and does not preclude other units from becoming active.

5.3. Implications for Gradient-Based Attacks

Gradient-based adversarial attacks such as HotFlip rely on persistent, high-magnitude gradients with respect to token embeddings to identify impactful discrete perturbations. The preceding analysis suggests that monotonicity constraints can reduce the availability of such gradients by driving subsets of hidden units into saturated regimes from which they do not recover under monotone increases in internal representations.

While monotone feed-forward networks are not dynamical systems in the formal sense, their behavior is reminiscent of classical results in monotone dynamical systems, where co-operative interactions restrict the effective directions along which trajectories evolve. This analogy is intended as intuition rather than a formal guarantee: monotonicity does not prevent all adversarial attacks, nor do gradients vanish globally. Instead, our analysis identifies a local attenuation effect that weakens gradient-based optimization strategies, offering a plausible mechanism for the empirical robustness improvements observed in Section 6.

6. Results

We evaluate the impact of monotonicity constraints on both task performance and adversarial robustness. We first analyze optimization behavior and summarization quality on the encoder-decoder (T5) track to assess potential performance degradation, and then evaluate robustness under targeted adversarial attacks. We conclude with the decoder-only foundation model track (Section 6.3), which tests whether the robustness benefit transfers across architectures. Together, these experiments assess whether monotonicity serves as a practical inductive bias for improving robustness without sacrificing model utility.

6.1. Training Dynamics and Summarization Quality

We begin by analyzing the effect of monotonicity constraints on optimization behavior and summarization quality. Table 1 summarizes the training dynamics of the baseline and monotonic models from our experiments. The baseline model refers to a T5-small checkpoint fine-tuned under identical settings as the monotonic model but without monotonicity constraints, while the standard model denotes the pretrained T5-small without additional fine-tuning. The monotonic model starts with a substantially higher initial loss (4.97 vs. 2.94), reflecting the impact of the softplus reparameterization and the constrained semantic-space parameterization of g . This initialization disrupts the original weight geom-

Table 1. Training dynamics comparison (seed = 42).

Model	Initial Loss	Final Loss	Δ Loss	Epochs
Baseline	2.94	2.21	-0.73	6
Monotonic	4.97	2.54	-2.43	7

Table 2. Summarization quality on CNN/DailyMail (n = 200, seed = 42). Values are means with 95% bootstrap confidence intervals.

Model	ROUGE-1	ROUGE-2	ROUGE-L
Standard	32.6 [30.9, 34.2]	11.9 [10.5, 13.4]	26.6 [25.0, 28.1]
Baseline	30.9 [29.4, 32.4]	11.5 [10.1, 12.8]	25.0 [23.6, 26.4]
Monotonic	29.6 [28.0, 31.1]	10.6 [9.2, 11.9]	24.2 [22.7, 25.6]

etry, requiring the model to re-establish effective internal representations during fine-tuning.

Despite this unfavorable initialization, the monotonic model converges reliably during training. Validation loss decreases from 2.92 at epoch 1 to 2.54 at epoch 7, indicating stable optimization under the imposed constraints. The model exhibits rapid initial recovery, reducing training loss by 1.92 points in the first two epochs (from 4.97 to 3.05), followed by slower asymptotic convergence with diminishing returns in later epochs. A persistent optimization gap of 0.32 points remains at convergence (2.54 vs. 2.21 for baseline), consistent with the reduced expressivity of the constrained parameter space. This gap reflects a deliberate and quantifiable trade-off: the baseline model achieves slightly lower validation loss through unrestricted FFN weights, while the monotonic model maintains competitive performance while enforcing structural regularity that, as shown below, translates to improved adversarial robustness.

Table 2 reports summarization performance on CNN/DailyMail (n=200, seed 42). The monotonic model achieves a ROUGE-L score of 24.2 [22.7, 25.6], compared to 25.0 [23.6, 26.4] for the baseline, corresponding to a relative decrease of 3.2 percent. ROUGE-1 exhibits a similar trend (29.6 [28.0, 31.1] vs 30.9 [29.4, 32.4], a 4.2 percent decrease). While these results indicate a modest reduction in task performance, the monotonic model remains competitive, with confidence intervals showing substantial overlap across all metrics.

To assess generalization, we additionally evaluate on XSUM (n=200, seed 42). Table 3 shows consistent results: the monotonic model achieves ROUGE-L of 12.9 [12.3, 13.6] compared to 13.3 [12.6, 13.9] for baseline, a similar 3.0 percent relative gap. The consistency of the performance trade-off across distinct summarization datasets (news articles in CNN/DM, single-sentence summaries in XSUM) suggests that the cost of monotonicity constraints is stable and predictable across different data distributions.

The training dynamics exhibit a characteristic pattern of

Table 3. Summarization quality on XSUM (n = 200, seed = 42). Values are means with 95% bootstrap confidence intervals.

Model	ROUGE-1	ROUGE-2	ROUGE-L
Standard	19.9 [18.8, 21.1]	2.9 [2.3, 3.4]	13.3 [12.6, 14.1]
Baseline	20.0 [19.0, 21.0]	3.2 [2.7, 3.7]	13.3 [12.6, 13.9]
Monotonic	18.9 [18.0, 19.7]	2.8 [2.3, 3.2]	12.9 [12.3, 13.6]

monotone-constrained optimization. During the first two epochs, the monotonic model rapidly recovers much of the performance gap introduced by constrained initialization, reducing training loss by 1.92 points (from 4.97 to 3.05). Subsequent epochs yield diminishing returns, with only 0.36 points of additional improvement from epochs 3 through 7. This behavior highlights the importance of the extended warmup phase (15 percent vs 10 percent for baseline), which stabilizes gradient estimates and enables effective learning despite the constrained parameter space.

Analysis of generation behavior reveals that both fine-tuned models produce longer outputs than the pretrained T5 model (mean lengths 74 to 76 tokens vs 57 tokens), reflecting adaptation to the distributional properties of the summarization training data. The monotonic model exhibits slightly higher length variance (std 11.3 tokens) compared to the standard model (std 12.0 tokens), though this difference is minimal. Importantly, the brevity penalty for the monotonic model (1.00) indicates that output length is well-calibrated to reference summary length, suggesting the constraints do not introduce systematic length bias.

6.2. Adversarial Robustness

HotFlip Attacks. We evaluate gradient-based robustness using HotFlip attacks, which identify vulnerable input positions via gradients with respect to token embeddings and perform targeted token substitutions to maximize loss. For each example, the attack computes gradients of the cross-entropy loss, selects the five positions with largest gradient magnitudes, and replaces tokens at those positions with vocabulary items that maximize the dot product between the gradient and the replacement embedding.

Robustness is quantified using three complementary metrics: average degradation, the mean relative loss increase under attack; attack success rate, the fraction of examples for which degradation exceeds 10%; and mean loss increase, the average absolute loss increase.

Table 4 reports results on 100 test examples from seed 42. To verify robustness consistency, we repeated the HotFlip evaluation using the same checkpoints with two additional random seeds (1337, 2024) for attack sampling variability. Results were identical across all three seeds, confirming that the observed robustness improvements are stable. The monotonic model demonstrates improved robustness rela-

Table 4. HotFlip attack results on CNN/DailyMail (n = 100, seed = 42).

Model	Avg. Deg.	Success Rate	Δ Loss
Standard	21.3%	69%	+0.42
Baseline	16.2%	63%	+0.35
Monotonic	5.2%	19%	+0.14

tive to both baselines. Under HotFlip perturbations, the monotonic model incurs an average degradation of 5.2 percent, compared to 16.2 percent for the fine-tuned baseline and 21.3 percent for the standard pretrained model. Attack success rates follow a similar pattern, with only 19 percent of attacks succeeding against the monotonic model, compared to 63 percent for the baseline and 69 percent for the standard model. This corresponds to a 69.8 percent relative reduction in attack success rate (63 percent to 19 percent) and a 67.9 percent reduction in average degradation (16.2 percent to 5.2 percent). Paired t -tests confirm that the difference between the baseline and monotonic models is highly significant ($t=6.17$, $p = 3.8 \times 10^{-9}$), with an even larger effect relative to the standard model ($t=7.45$, $p=2.8 \times 10^{-12}$).

Overall, enforcing monotonicity in the FFN sublayers yields a 69.8 percent relative reduction in attack success rate (from 63 percent to 19 percent) and a 67.8 percent reduction in average degradation (from 16.1 percent to 5.2 percent), without requiring adversarial training, modified training objectives, or architectural changes beyond the FFN constraints. The effect size is large (Cohen’s $d > 0.8$) and the statistical significance is robust ($p < 10^{-9}$). These results demonstrate that structural constraints alone—applied selectively to feed-forward sublayers—can substantially improve robustness to gradient-based adversarial manipulation, providing evidence for monotonicity as a practical architectural inductive bias for building more robust language models.

Universal Adversarial Trigger Attacks. We evaluate robustness to universal adversarial triggers (UATs), input-agnostic token sequences optimized to maximize model loss when prepended to any input. Triggers are learned via coordinate ascent with 3 restarts and 50 iterations over 80 training examples, then evaluated on 120 disjoint test examples. Robustness is measured via NLL increase under trigger prepending. UAT attacks prove minimally effective across all models, with NLL increases below 1 percent. The monotonic model exhibits the smallest degradation (0.73 percent NLL increase), compared to 0.89 percent for the baseline and 0.97 percent for standard T5. While this ordering is consistent with improved robustness, the effect sizes are too small to draw strong conclusions. The weak effectiveness of universal triggers, in contrast to the impact of input-specific HotFlip attacks, suggests that input-agnostic perturbations are limited for attacking sequence-to-sequence models, and

Table 5. Constraint-scope screening on Pythia-1.4B (seed = 42). PPL denotes held-out Pile perplexity; SR denotes HotFlip attack success rate ($n = 200$, 10 flips). MLP-in was screened under the full training budget, the other scopes under a reduced budget.

Constraint scope	PPL base	PPL mono	SR base $\rightarrow$ mono	SR drop
MLP-in	6.67	24.32	71.5% $\rightarrow$ 83.5%	−12.0 pp
MLP-both	7.06	47.99	75.0% $\rightarrow$ 44.5%	+30.5 pp
MLP-in + Attn-out	7.06	88.17	75.0% $\rightarrow$ 35.5%	+39.5 pp

that monotonicity’s robustness benefits are most pronounced against adaptive, gradient-based attacks.

6.3. Generalization to Decoder-Only Foundation Models

We next ask whether the robustness benefit of monotone feed-forward constraints is specific to the encoder-decoder setting or extends to decoder-only foundation models, using the Pythia-1.4B track described in Section 4.2.

Constraint scope determines whether robustness transfers. Table 5 reports the constraint-scope screening. The direct analogue of the T5 scheme—constraining only the MLP input projection (MLP-in)—fails on the decoder-only architecture: perplexity increases by a factor of 3.65, yet the HotFlip attack success rate *rises* from 71.5% to 83.5%. The constraint degrades clean performance without impeding the attack, indicating that adversarial gradient signal is routed around the constrained matrix through the unconstrained MLP output projection and attention pathways. Extending the constraint scope closes these leak paths: constraining both MLP projections (MLP-both) reduces the success rate by 30.5 percentage points, and additionally constraining the attention output projection (MLP-in + Attn-out) yields a 39.5-point reduction. The latter, however, roughly doubles the perplexity penalty ($12.49\times$ versus $6.80\times$) for a further 9-point reduction in success rate, a diminishing return consistent with attention serving as a secondary leak path. Balancing robustness against clean performance, we select MLP-both for full-budget validation.

Multi-seed validation. Table 6 reports the MLP-both scope under the full recovery-training budget across three seeds. The robustness improvement is stable: the HotFlip success rate falls from 69.0% to between 39.0% and 45.0% (mean 42.5%), an absolute reduction of 26.5 percentage points and a relative reduction of 38.4%. Under UAT attacks—which, unlike on T5, are substantially damaging to the decoder-only baseline, raising NLL by approximately 16%—the monotone models are consistently less affected, with a mean NLL increase of 10.6% versus 16.1% for the baseline. The monotone models thus improve robustness against both adaptive gradient-based and input-agnostic attacks in this setting.

The quality–robustness trade-off is architecture-dependent. The robustness gains on Pythia-1.4B come at a markedly higher cost in clean performance than on

Table 6. Pythia-1.4B results with the MLP-both constraint scope under the full recovery budget. PPL denotes held-out Pile perplexity; SR denotes HotFlip attack success rate ($n = 200$, 10 flips); UAT NLL $\uparrow$ denotes the relative NLL increase under the learned universal trigger (base $\rightarrow$ monotone).

Seed	PPL base	PPL mono	SR base $\rightarrow$ mono	UAT NLL $\uparrow$
42	6.72	36.01	69.0% $\rightarrow$ 43.5%	+15.5% $\rightarrow$ +8.8%
1337	6.72	42.00	69.0% $\rightarrow$ 39.0%	+16.4% $\rightarrow$ +8.1%
2024	6.72	35.03	69.0% $\rightarrow$ 45.0%	+16.5% $\rightarrow$ +15.0%
Mean	6.72	37.68	69.0% $\rightarrow$ 42.5%	+16.1% $\rightarrow$ +10.6%

T5. Held-out perplexity increases by a mean factor of 5.61 (range 5.21–6.25), and full-budget recovery training does not close this gap, whereas the T5 monotone model recovers to within a few percent of its baseline. We attribute this asymmetry to architectural structure: in T5, the encoder-decoder bottleneck and cross-attention alignment restrict the routes by which token-level perturbations can propagate without traversing a constrained sublayer, so constraining the feed-forward input projections suffices and leaves most of the network free to recover expressivity. In a decoder-only model, autoregressive self-attention connects every position to all preceding positions without passing through any constrained layer, providing a bypass for adversarial signal; robustness therefore requires constraining a larger fraction of each block’s transformation, which in turn imposes a steeper expressivity penalty. The monotonicity–robustness mechanism itself transfers across architectures, but the constraint scope required to realize it—and the price paid in clean performance—is architecture-conditional.

7. Discussion

This work provides empirical evidence and theoretical analysis supporting monotonicity as a structural bias for improving robustness in modern language models. By selectively enforcing monotonicity in feed-forward sublayers, we observe consistent reductions in vulnerability to adversarial and jailbreak-style attacks—while largely preserving task performance in the encoder-decoder setting. These gains arise without additional data, modified training objectives, or heuristic defenses, highlighting architectural structure as a direct lever for shaping model behavior under perturbation.

Our theoretical analysis offers insight into why these gains arise, even in the absence of formal end-to-end robustness guarantees. Viewing a language model as a high-dimensional system whose internal semantic state evolves through successive nonlinear transformations, monotonicity constrains how perturbations can propagate across layers. In particular, directional consistency ensures that strengthening information or constraints in internal representations cannot induce regressions downstream. This restriction narrows

the range of admissible behaviors and simplifies reasoning about worst-case effects, which must occur at semantic extremes rather than intermediate variations.

Empirically, the robustness gains are closely tied to the role of feed-forward sublayers. While attention mechanisms perform semantic aggregation and contextual reasoning, FFNs dominate nonlinear transformation and amplification. Enforcing monotonicity at this stage constrains how semantic information is refined after context has been established, without interfering with attention or global information flow. This selective enforcement alters the local gradient structure of the network, reducing the stability and magnitude of exploitable gradients used by attacks such as HotFlip.

These observations align with classical insights from monotone systems theory, where structural monotonicity limits perturbation propagation and enables tractable analysis of global behavior in high-dimensional systems. Although Transformer models are not dynamical systems in a formal sense, the analogy provides intuition and motivates our analysis of monotonic components within the network.

The decoder-only experiments sharpen this picture by showing that the leak-path argument is architectural rather than incidental. Where the encoder-decoder structure of T5 funnels perturbations through constrained sublayers, the residual stream of a decoder-only model offers unconstrained routes around any single constrained matrix; constraining only the feed-forward input projection accordingly fails on Pythia-1.4B, while constraining both feed-forward projections restores the robustness benefit. Monotonicity should therefore be understood not as a property of an isolated sublayer but of the set of paths through which adversarial signal can propagate—an observation that directly informs where constraints must be placed in a given architecture.

In the encoder-decoder setting, the trade-off introduced by monotonicity is modest and predictable: constraining FFN weights slightly reduces expressivity, and the resulting decrease in summarization quality is small relative to the robustness improvements. In the decoder-only setting, the broader constraint coverage required for robustness carries a substantially larger perplexity cost that full-budget recovery training does not eliminate, and reducing this cost—through better initialization, specialized training schedules, or partial-layer application—is a concrete open problem. More broadly, monotonicity differs from conventional regularization in that it constrains *how* representations may change, rather than simply penalizing their magnitude, promoting predictability rather than uniform smoothness.

Our study is limited to models of modest scale (T5-small and Pythia-1.4B) and does not provide formal robustness certificates. Extending the theoretical analysis toward stronger guarantees, scaling the approach to larger models, and nar-

rowing the quality gap on decoder-only architectures are important directions for future work. Nonetheless, the combination of analytical insight and consistent empirical gains across two architecture families suggests that monotonicity is a principled and scalable design choice for improving the robustness and predictability of large language models.

8. Conclusion

We investigated monotonicity as a structural inductive bias for improving robustness in Transformer language models. By enforcing non-negativity constraints on feed-forward sublayers, we observe substantial reductions in vulnerability to gradient-based adversarial attacks with only modest impact on summarization quality in the encoder-decoder setting. Extending the study to a decoder-only foundation model shows that the benefit transfers, but only when the constraint scope is matched to the architecture: decoder-only models require constraining both feed-forward projections to close adversarial leak paths, and incur a larger cost in clean perplexity. These results suggest that simple architectural constraints can meaningfully improve robustness without altering attention mechanisms or relying on adversarial training, provided constraint placement accounts for how signal propagates through the architecture.

An important direction for future work is to develop theoretical guarantees that characterize when and why monotonicity yields improved robustness in high-dimensional language models. Reducing the perplexity cost of monotone constraints on decoder-only architectures, and scaling this approach to larger model variants and broader tasks, are further promising avenues that may help clarify the role of structural constraints in building more reliable language models.

Impact Statement

This work explores architectural constraints that improve the robustness of language models to adversarial manipulation. By studying monotonicity as a structural inductive bias, our approach contributes to the development of language models whose behavior is more predictable and analyzable, which is particularly relevant for safety-critical and decision-support applications.

References

- Amos, B. and Kolter, J. Z. Optnet: Differentiable optimization as a layer in neural networks. In *International Conference on Machine Learning (ICML)*, 2017.
- Anil, C., Durmus, E., Panickssery, N., Sharma, M., Benton, J., Kundu, S., Batson, J., Tong, M., Mu, J., Ford, D., et al. Many-shot jailbreaking. *Advances in Neural Information*

- Processing Systems*, 37:129696–129742, 2024.
- Biderman, S., Schoelkopf, H., Anthony, Q. G., Bradley, H., O’Brien, K., Hallahan, E., Khan, M. A., Purohit, S., Prashanth, U. S., Raff, E., Skowron, A., Sutawika, L., and van der Wal, O. Pythia: A suite for analyzing large language models across training and scaling. In *Proceedings of the 40th International Conference on Machine Learning*, volume 202, pp. 2397–2430. PMLR, 2023.
- Cao, B., Cao, Y., Lin, L., and Chen, J. Defending against alignment-breaking attacks via robustly aligned llm. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pp. 10542–10560, 2024.
- Daniels, H. and Velikova, M. Monotone and partially monotone neural networks. *IEEE Transactions on Neural Networks*, 21(6):906–917, 2010.
- Deng, Y., Zhang, W., Pan, S. J., and Bing, L. Multilingual jailbreak challenges in large language models. 2023. *ArXiv, abs/2310.06474*, 2024.
- Ebrahimi, J., Rao, A., Lowd, D., and Dou, D. HotFlip: White-box adversarial examples for text classification. In *Proceedings of the 56th Annual Meeting of the Association for Computational Linguistics (Volume 2: Short Papers)*, pp. 31–36. Association for Computational Linguistics, 2018.
- Feyzmahdavian, H. R., Besselink, B., and Johansson, M. Stability analysis of monotone systems via max-separable lyapunov functions. *IEEE Transactions on Automatic Control*, 63(3):643–656, 2017.
- Gao, L., Biderman, S., Black, S., Golding, L., Hoppe, T., Foster, C., Phang, J., He, H., Thite, A., Nabeshima, N., Presser, S., and Leahy, C. The Pile: An 800GB dataset of diverse text for language modeling. *arXiv preprint arXiv:2101.00027*, 2020.
- Gupta, M., Cotter, A., Pfeifer, J., and Voevodski, K. Monotonic calibrated interpolated look-up tables. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2016.
- Hirsch, M. W. Systems of differential equations that are competitive or cooperative ii: Convergence almost everywhere. *SIAM Journal on Mathematical Analysis*, 16(3): 423–439, 1985.
- Howe, N. H., McKenzie, I. R., Hollinsworth, O. J., Zajac, M., Tseng, T., Tucker, A. D., Bacon, P.-L., and Gleave, A. Effects of scale on language model robustness. 2024.
- Ito, H., Rüffer, B. S., and Rantzer, A. Max-and sum-separable lyapunov functions for monotone systems and their level sets. In *53rd IEEE Conference on Decision and Control*, pp. 2371–2377. IEEE, 2014.
- Jukna, S. et al. *Boolean function complexity: advances and frontiers*, volume 27. Springer, 2012.
- Khachaturov, D. and Mullins, R. Adversarial suffix filtering: a defense pipeline for llms. *arXiv preprint arXiv:2505.09602*, 2025.
- Langley, P. Crafting papers on machine learning. In Langley, P. (ed.), *Proceedings of the 17th International Conference on Machine Learning (ICML 2000)*, pp. 1207–1216, Stanford, CA, 2000. Morgan Kaufmann.
- Li, M. Q. and Fung, B. C. Security concerns for large language models: A survey. *Journal of Information Security and Applications*, 95:104284, 2025.
- Michel, A. N., Hou, L., and Liu, D. *Stability of dynamical systems: On the role of monotonic and non-monotonic Lyapunov functions*. Springer, 2015.
- Newhouse, L., Hess, R. P., Cesista, F., Zahorodnii, A., Bernstein, J., and Isola, P. Training transformers with enforced lipschitz constants. *arXiv preprint arXiv:2507.13338*, 2025.
- Nguyen, A.-P., Moreno, D. L., Le-Bel, N., and Rodríguez Martínez, M. Mononet: enhancing interpretability in neural networks via monotonic features. *Bioinformatics advances*, 3(1):vbad016, 2023.
- Raffel, C., Shazeer, N., Roberts, A., Lee, K., Narang, S., Matena, M., Zhou, Y., Li, W., and Liu, P. J. Exploring the limits of transfer learning with a unified text-to-text transformer. *Journal of machine learning research*, 21 (140):1–67, 2020.
- Rantzer, A. and Bernhardsson, B. Control of convex-monotone systems. In *53rd IEEE Conference on Decision and Control*, pp. 2378–2383. IEEE, 2014.
- Robey, A., Wang, Z., and Kolter, J. Z. Smoothllm: Defending large language models against jailbreak attacks. *arXiv preprint arXiv:2310.03684*, 2023.
- Runje, D. and Shankaranarayana, S. M. Constrained monotonic neural networks. In *International Conference on Machine Learning*, pp. 29338–29353. PMLR, 2023.
- Salah, A. R. M. Jailbreaking llms: An in-depth security analysis of prompt injection vulnerabilities. 2026.
- Sartor, D., Sinigaglia, A., and Susto, G. A. Advancing constrained monotonic neural networks: Achieving universal approximation beyond bounded activations. *arXiv preprint arXiv:2505.02537*, 2025.

- Sayeedi, M. F. A., Hossain, M. B., Hassan, M. K., Afrin, S., Sabit, M. M., and Hossain, M. S. Jailbreaktracer: Explainable detection of jailbreaking prompts in llms using synthetic data generation. *IEEE Access*, 2025.
- Sill, J. Monotonic networks. In *Advances in Neural Information Processing Systems (NeurIPS)*, 1998.
- Smith, H. L. *Monotone dynamical systems: an introduction to the theory of competitive and cooperative systems*. Number 41. American Mathematical Soc., 1995.
- Su, J., Kempe, J., and Ullrich, K. Mission impossible: A statistical perspective on jailbreaking llms. *Advances in Neural Information Processing Systems*, 37:38267–38306, 2024.
- Wallace, E., Feng, S., Kandpal, N., Gardner, M., and Singh, S. Universal adversarial triggers for attacking and analyzing NLP. In *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing (EMNLP-IJCNLP)*, pp. 2153–2162. Association for Computational Linguistics, 2019.
- Wang, A., Singh, A., Michael, J., Hill, F., and Bowman, S. R. Adversarial glue: A multi-task benchmark for robustness evaluation of language models. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2021.
- Wang, H., Wang, J., Zhao, Z., Tan, Y., Wu, Y., Liu, H., Yang, J., Zhang, E., Chen, X., Rong, Z., et al. Understanding knowledge transferability for transfer learning: A survey. *arXiv preprint arXiv:2507.03175*, 2025a.
- Wang, Y., Chen, M., Peng, N., and Chang, K.-W. Vulnerability of large language models to output prefix jailbreaks: Impact of positions on safety. In *Findings of the Association for Computational Linguistics: NAACL 2025*, pp. 3939–3952, 2025b.
- Weber, M., Balestrieri, R., and Baraniuk, R. Certifying monotonic neural networks. *arXiv preprint arXiv:2102.12566*, 2021.
- You, S., Ding, Y., Canini, M., Pfeifer, J., and Gupta, M. Deep lattice networks and partial monotonic functions. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2017.
- Zhu, Y., Xie, Y., Chen, X., and Zhang, Q. Autodan: Generating stealthy jailbreak prompts for large language models. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2023.
- Zou, A., Wang, Z., Carlini, N., Nasr, M., and Kolter, J. Z. Universal and transferable adversarial attacks on aligned language models. *arXiv preprint arXiv:2307.15043*, 2023.

A. Appendices

A.1. Mathematical Proofs

A.1.1. PROOF OF LEMMA 3.5

Proof. For any $x, x' \in \mathbb{R}^n$ such that $x \leq x'$, monotonicity of f implies $f(x) \leq f(x')$. Applying monotonicity of g yields

$$g(f(x)) \leq g(f(x')),$$

which establishes the claim. $\square$

A.1.2. PROOF OF LEMMA 5.1

Proof. Recall $T(s) = s + g(s)$, so

$$J_T(s) = \nabla_s T(s) = I + J_g(s).$$

For a two-layer $g(s) = W_2 \sigma(W_1 s + b_1) + b_2$ with $W_1, W_2 \geq 0$ elementwise,

$$J_g(s) = W_2 \text{diag}(\sigma'(W_1 s + b_1)) W_1.$$

Since σ is elementwise non-decreasing, $\sigma'(z) \geq 0$ wherever it exists, so $\text{diag}(\sigma'(\cdot)) \geq 0$ elementwise. Hence $J_g(s) \geq 0$ elementwise, and therefore $J_T(s) = I + J_g(s) \geq 0$ elementwise. $\square$

A.1.3. PROOF OF LEMMA 5.4

Proof. Let $z_j(s) := (W_1 s + b_1)_j$. Since $W_1 \geq 0$ elementwise and $\delta \geq 0$,

$$z_j(s' + \delta) = (W_1(s' + \delta) + b_1)_j \geq (W_1 s' + b_1)_j = z_j(s').$$

Thus if the unit is saturated at s' (i.e., $\sigma'(z_j(s')) = 0$ in the saturated regime), it remains saturated at $s' + \delta$. $\square$

A.2. Training and Evaluation Protocol

Both baseline and monotone models are fine-tuned from the same pretrained T5-small checkpoint under identical optimization settings. We use the AdamW optimizer with learning rate 5×10^{-5} and weight decay 0.01, a batch size of 4, and gradient clipping with norm 1.0. To accommodate the constrained parameterization, the monotone model employs an extended warmup phase of 15% of training steps, compared to 10% for the baseline. Both models are trained for 7 epochs and receive the same total training budget.

Training is conducted on the DialogSum, HighlightSum, and arXiv abstract datasets, comprising approximately 150K examples in total. Evaluation is performed on three held-out benchmarks: CNN/DailyMail (11,490 test examples), XSUM (11,334 test examples), and SAMSum (819 test examples). CNN/DailyMail is excluded entirely from training to assess out-of-distribution generalization.

At inference time, decoding is performed using beam search with 4 beams and a length penalty of 1.2, enforcing a minimum generation length of 10 tokens and a maximum length of 80 tokens. We additionally apply no-repeat n -gram blocking with $n = 3$ to discourage degenerate repetitions. All decoding parameters are fixed across models and evaluation sets. ROUGE-1, ROUGE-2, and ROUGE-L scores are computed using the rouge-score library with Porter stemming. We report bootstrap-based 95% confidence intervals using 1,000 resamples and assess statistical significance via paired t -tests with Bonferroni correction for multiple comparisons. Effect sizes are reported using Cohen’s d .

A.3. Experimental Setup

To evaluate robustness to training stochasticity, all experiments are repeated using five random seeds (42, 1337, 2024, 8888, and 12345), controlling randomness across Python, NumPy, and PyTorch. We report the mean and standard deviation across seeds for all metrics. Statistical significance across training runs is assessed using paired t -tests on seed-level means, complemented by per-example paired t -tests within each seed. This dual evaluation captures both test-time variability and variance induced by random initialization and data ordering. All experiments are conducted using deterministic algorithms where available, on a single NVIDIA A100 GPU with 40GB memory, using PyTorch 2.0.1 and Transformers 4.30.2.

A.4. Foundation Model Training Protocol

The decoder-only track fine-tunes Pythia-1.4B (Biderman et al., 2023) on the uncopyrighted subset of the Pile (Gao et al., 2020), with held-out evaluation texts drawn from a disjoint streaming sample. Constraint-scope screening (Table 5) uses a reduced budget of 30,000 training samples at sequence length 1024 for 3–4 epochs on seed 42; full-budget validation of the selected MLP-both scope (Table 6) trains the baseline for 5 epochs and the monotone model for 10 epochs on each of three seeds (42, 1337, 2024). The extended monotone budget parallels the extended warmup used in the T5 track and compensates for the constrained parameterization’s slower recovery. HotFlip evaluation uses 200 held-out examples with up to 10 token flips per example; UAT evaluation uses coordinate ascent with 3 restarts and 50 iterations, optimizing triggers on 80 texts and evaluating on 120 disjoint texts. Attack-time model evaluation is performed in bfloat16 precision. All foundation-model experiments are executed on NVIDIA A100 GPUs with 80GB memory under the SLURM workload manager, with per-stage checkpointing to permit resumption across scheduler preemptions.